



誠朴雄偉
勵學敦行

复习课（四）

（中间代码表示）

冯洋

南京大学计算机学院

fengyang@nju.edu.cn





中间代码表示



五、简答题（共 1 题，共 10 分）

1. (5 分) 在图 1 中间代码翻译方案中添加处理如下控制流构造的规则（注意布尔表达式之前的否运算）：

$S \rightarrow \text{for } (S_1; !B; S_2) S_3$

2. (5 分) 结合图 2 布尔表达式的翻译方案，翻译如下代码（第 5 到第 7 行）：

要求：画出语法分析树，并在每个节点上标注相应的属性，给出最终的中间代码。

```
(1)  int i;  
(2)  int n = 100;  
(3)  int sum = 0;  
(4)  
(5)  for (i = 0; i < n && sum < 1000; i = i + 1) {  
(6)      sum = sum + i;  
(7)  }
```

得分	
评分人	

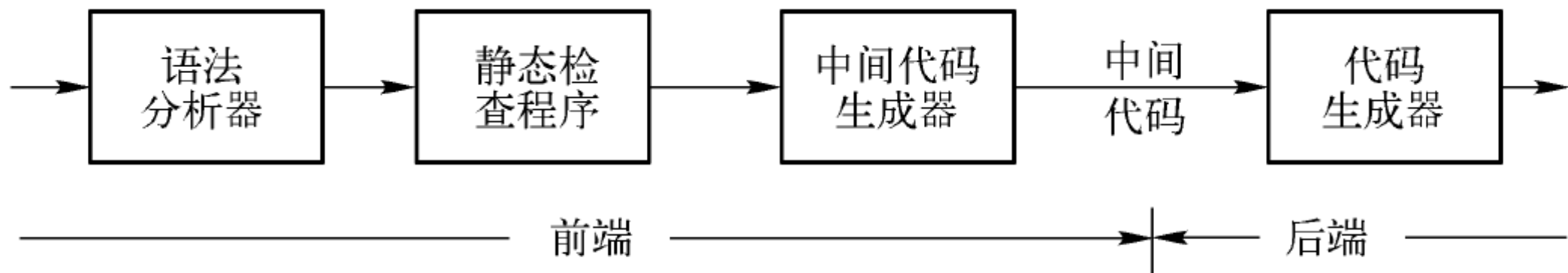
产生式	语义规则
$P \rightarrow S$	$S.next = \text{newlabel}()$ $P.code = S.code \parallel \text{label}(S.next)$
$S \rightarrow \text{assign}$	$S.code = \text{assign}.code$
$S \rightarrow \text{if } (B) S_1$	$B.true = \text{newlabel}()$ $B.false = S_1.next = S.next$ $S.code = B.code \parallel \text{label}(B.true) \parallel S_1.code$
$S \rightarrow \text{if } (B) S_1 \text{ else } S_2$	$B.true = \text{newlabel}()$ $B.false = \text{newlabel}()$ $S_1.next = S_2.next = S.next$ $S.code = B.code$ $\parallel \text{label}(B.true) \parallel S_1.code$ $\parallel \text{gen}('goto' S.next)$ $\parallel \text{label}(B.false) \parallel S_2.code$
$S \rightarrow \text{while } (B) S_1$	$begin = \text{newlabel}()$ $B.true = \text{newlabel}()$ $B.false = S.next$ $S_1.next = begin$ $S.code = \text{lable}(begin) \parallel B.code$ $\parallel \text{label}(B.true) \parallel S_1.code$ $\parallel \text{gen}('goto' begin)$
$S \rightarrow S_1 S_2$	$S_1.next = \text{newlable}()$ $S_2.next = S.next$ $S.code = S_1.code \parallel \text{label}(S_1.next) \parallel S_2.code$



中间代码表示



- 中间表示（Intermediate Representation, IR）:编译器在前端与后端之间使用的、与具体源语言和目标机器都相对独立的程序表示形式，用于承载程序语义、驱动优化，并作为目标代码生成的基础。





中间代码表示



- 高层IR（高抽象、语义保留多）
 - 接近源语言结构：函数、循环、异常、对象、闭包等仍显式存在
 - 优点：便于高层语义相关优化（如内联、异常消除、对象去虚拟化）
 - 缺点：与目标机器距离远，难以直接选址与寄存器分配
- 中层IR（常见）
 - 典型：三地址码（TAC）、四元式、SSA形态的TAC
 - 权衡：足够接近机器以便分析，又保留必要结构信息
- 低层IR（低抽象、接近机器）
 - 显式寄存器/栈、分支、加载/存储、调用约定
 - 优点：易映射到目标指令；缺点：优化空间受限



中间代码表示



■ IR设计与分析的关键维度

○ 控制流表示

- 结构化（如高层循环构造） vs 显式跳转与标签（基本块图）

○ 数据表示

- 值模型：三地址临时 vs 栈操作数（堆栈式IR）
- 赋值模型：普通赋值 vs 静态单赋值（SSA，单点写，多点读）

○ 内存与地址

- 显式load/store vs 隐式求值
- 别名与指针是否可见、是否区分地址和值（l-value/r-value）

○ 类型与语义信息

- 强类型IR（带精确类型与对齐） vs 弱类型/无类型
- 高层语义保留程度（异常、越界检查、GC屏障、并发原语）

○ 可分析性与可变换性

- 是否便于构建CFG/DFG、做数据流分析、回填与短路求值



中间代码表示



■ 常见IR范式与代表

- 树形IR：抽象语法树（AST）、语法树-语义注释
 - 适合“早期”高层变换，不适合低层优化
- 三地址码（TAC）： $x = y \text{ op } z$; if x goto L; param x; call f, n; return x
 - 表示方式：四元式（op, arg1, arg2, result）、三元式（op, arg1, arg2）
- SSA（静态单赋值）形态的IR：在TAC基础上引入 Φ 函数
 - 优化友好：简化数据流问题，如活跃性、可用表达式、冗余消除
- 堆栈式IR：操作数隐含在栈上（如JVM字节码）
 - 简单、紧凑，但数据依赖不显式，部分优化不便
- LLVM IR/Sea-of-Nodes 等工业IR
 - 应用极其广泛



三地址码



- 定义与概述：三地址码是一种常见的中间表示形式，它将源程序转换为一系列简单的指令，每条指令最多涉及三个“地址”（操作数或结果）
- 典型格式为： $\text{result} = \text{arg1 op arg2}$ ，其中op是操作符，arg1和arg2是操作数，result是结果变量
 - “地址”可以是常量、变量、临时变量或标签
 - 它类似于低级汇编语言，但独立于具体机器架构，便于编译器后端处理



三地址码



- 定义与概述：三地址码是一种常见的中间表示形式，它将源程序转换为一系列简单的指令，每条指令最多涉及三个“地址”（操作数或结果）
- 在编译器中的位置：三地址码通常从抽象语法树（**AST**）生成，是前端（语法/语义分析）与后端（优化/目标代码生成）之间的桥梁
- 为什么叫“三地址”？因为大多数指令模拟三操作数机器的运算（如加法： $t1 = a + b$ ），相比更高级的**IR**（如树状表示），它更线性且易于翻译成机器码



三地址码



- 每条指令右侧最多有一个运算符
 - 一般情况可以写成 $x = y \text{ op } z$
- 允许的运算分量
 - 名字：源程序中的名字作为三地址代码的地址
 - 常量：源程序中出现或生成的常量
 - 编译器生成的临时变量



三地址码



■ 三地址码实例

语句

do $i = i + 1$; while ($a[i] < v$);

```
L:  t1 = i + 1  
    i = t1  
    t2 = i * 8  
    t3 = a [ t2 ]  
    if t3 < v goto L
```

a) 符号标号

```
100:  t1 = i + 1  
101:  i = t1  
102:  t2 = i * 8  
103:  t3 = a [ t2 ]  
104:  if t3 < v goto 100
```

b) 位置号



三地址码



- 在实现时，可以使用**四元式/三元式/间接三元式**来表示三地址指令
- 四元式：可以实现为纪录（或结构）
- 格式（字段）： $op \quad arg1 \quad arg2 \quad result$
 - op : 运算符的内部编码
 - $arg1, arg2, result$ 是地址
 - $x=y+z \quad \quad \quad + \quad y \quad z \quad x$
- 单目运算符不使用 $arg2$
- $param$ 运算不使用 $arg2$ 和 $result$
- 条件转移/非条件转移将目标标号放在 $result$ 字段



三地址码



- 四元式的实例
- 赋值语句: $a = b * -c + b * -c$

```
t1 = minus c
t2 = b * t1
t3 = minus c
t4 = b * t3
t5 = t2 + t4
a = t5
```

a) 三地址代码

	<i>op</i>	<i>arg₁</i>	<i>arg₂</i>	<i>result</i>
0	minus	c		t ₁
1	*	b	t ₁	t ₂
2	minus	c		t ₃
3	*	b	t ₃	t ₄
4	+	t ₂	t ₄	t ₅
5	=	t ₅		a
	...			

b) 四元式



三地址码



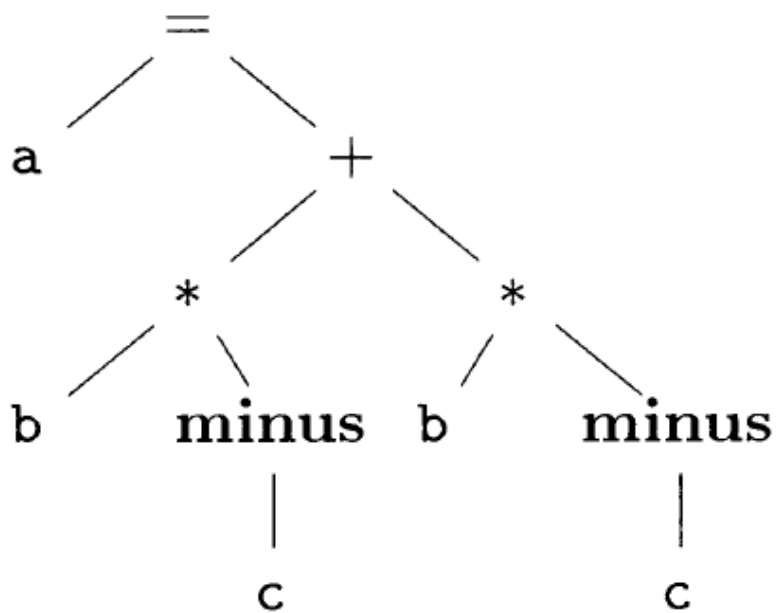
- 三元式表示 $\text{triple op arg1 arg2}$
- 使用三元式的位置来引用三元式的运算结果
- $x[i]=y$ 需要拆分为两个三元式
 - 求 $x[i]$ 的地址，然后再赋值
- $x=y \text{ op } z$ 需要拆分为（这里？是编号）
 - $(?) \text{ op } y \quad z$
 - $= \quad x \quad ?$
- 问题：在优化时经常需要移动/删除/添加三元式，导致三元式的移动，应该如何处理？



三地址码



■ 三元式的实例



a) 语法树

	<i>op</i>	<i>arg₁</i>	<i>arg₂</i>
0	minus	c	
1	*	b	(0)
2	minus	c	
3	*	b	(2)
4	+	(1)	(3)
5	=	a	(4)
	...		

b) 三元式



三地址码



- 间接三元式：包含了一个指向三元式的指针的列表
- 我们可以对这个列表进行操作，完成优化功能；操作时不需要修改三元式中的参数

instruction

35	(0)
36	(1)
37	(2)
38	(3)
39	(4)
40	(5)
	...

op *arg₁* *arg₂*

0	minus	c	
1	*	b	(0)
2	minus	c	
3	*	b	(2)
4	+	(1)	(3)
5	=	a	(4)
		...	



三地址码



■ 三地址码的指令类型

○ 算术/逻辑运算：用于表达式计算

■ 格式： $x = y \text{ op } z$ (op 如 +, -, *, /, &&, || 等)

■ 示例： $t1 = a + b$ (加法)； $t2 = t1 * c$ (乘法，使用临时变量链式处理)

○ 赋值与拷贝：简单数据移动

■ 格式： $x = y$ (单地址) 或 $x = \text{unary_op } y$ (如 $x = -y$)

■ 示例： $\text{sum} = 0$ (初始化)

○ 控制流指令：处理分支和循环

■ 无条件跳转： $\text{goto } L$ (L为标签)

■ 条件跳转： $\text{if } x \text{ relop } y \text{ goto } L$ (relop 如 <, >, ==)

■ 示例： $\text{if } t1 \leq 10 \text{ goto } L1$ (循环条件)



三地址码



■ 三地址码的指令类型

○ 函数调用与参数：处理过程调用

- 格式：param x (传递参数); call func, n (调用函数, n参数个数); x = call func (带返回值)
- 示例：param sum; call printf, 1

○ 数组/指针访问：地址计算

- 格式：x = y[i] 或 x[i] = y; 需计算偏移, 如 t1 = i * width; x = y + t1
- 示例：对于数组a[5], t1 = i * 4; t2 = a + t1; val = t2 (假设int类型4字节)

○ 标签：标记代码位置

- 格式：L: (



三地址码



■ 随堂测试题目： $x = (a + b) * (c - d) + e$

写出这个式子的三元式、间接三元式、四元式和AST树



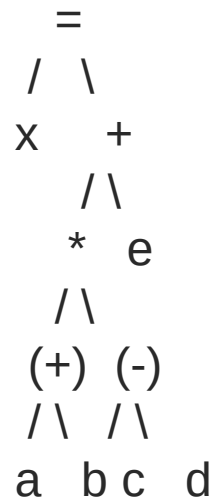
三地址码



■ 随堂测试题目： $x = (a + b) * (c - d) + e$

写出这个式子的三元式、间接三元式、四元式和AST树

序号	op	arg1	arg2	result
(1)	+	a	b	t1
(2)	-	c	d	t2
(3)	*	t1	t2	t3
(4)	+	t3	e	t4
(5)	=	t4	-	x





三地址码



■ 随堂测试题目： $x = (a + b) * (c - d) + e$

写出这个式子的三元式、间接三元式、四元式和AST树

序号	op	arg1	arg2	序号	op	arg1	arg2
(1)	+	a	b	(1)	+	a	b
(2)	-	c	d	(2)	-	c	d
(3)	*	(1)	(2)	(3)	*	(0)	(1)
(4)	+	(3)	e	(4)	+	(2)	e
(5)	=	(4)	x	(5)	=	(3)	x

0 → (1)

1 → (2)

2 → (3)

3 → (4)

4 → (5)

(注意：这里 (0) 和 (1) 表示索引表里的引用，而不是直接的三元式编号，便于优化和重排。)



静态单赋值



- 静态单赋值形式（Static Single Assignment, SSA）是编译器设计中用于优化的中间表示形式，其核心规则要求每个变量仅被赋值一次，通过添加下标区分变量不同版本
- 静态单赋值形式被广泛应用于中间代码表示，使用SSA可以很容易地发现一些可优化项，如死代码优化、常量传播等
- 优点：
 - 消除了变量重赋值带来的混淆
 - 数据流清晰，每个使用点对应唯一的定义点
 - 大大简化编译器的优化（如常量传播、死代码删除、寄存器分配）



静态单赋值



- 1981 年：由 Cytron 等人在论文 Efficiently Computing Static Single Assignment Form and the Control Dependence Graph 中提出
 - 初衷：为优化数据流分析，引入“单赋值”思想，但控制流合流点需要额外机制，因此发明了 ϕ 函数
 - 后续：SSA 成为工业级编译器的核心技术（LLVM、GCC、Java HotSpot、Rustc 都使用 SSA 或变体）



静态单赋值



- SSA中的所有赋值都是针对不同名的变量
- 对于同一个变量在不同路径中定值的情况，可以使用 ϕ 函数来合并不同的定值

```
if (flag)
    x = -1;
else
    x = 1;

y = x*a
```



静态单赋值



- SSA中的所有赋值都是针对不同名的变量
- 对于同一个变量在不同路径中定值的情况，可以使用 ϕ 函数来合并不同的定值

```
if (flag)
    x = -1;
else
    x = 1;
```

```
y = x*a
```

```
if (flag)
    x1=-1;
else
    x2 = 1;
```

```
x3= $\phi$ (x1,x2);
y = x3*a
```



静态单赋值



- SSA中的所有赋值都是针对不同名的变量
- 对于同一个变量在不同路径中定值的情况，可以使用 ϕ 函数来合并不同的定值

```
if (flag)
    x = -1;
else
    x = 1;
```

```
y = x*a
```

```
if (flag)
    x1 = -1;
else
    x2 = 1;
```

```
x3 =  $\phi$ (x1, x2);
y = x3*a
```



静态单赋值



■ φ 函数的本质

- 数学意义：一个“多路选择器”（类似 switch）。
- 语义：在控制流合流点，根据 控制路径 决定选哪个值。
- 形式：
$$x_{n+1} = \varphi(x_1, x_2, \dots, x_n)$$
- φ 不是 **CPU 指令**，而是编译器 IR 的“抽象伪指令”。
在生成汇编代码时，**必须把它消除（ φ -elimination）**



静态单赋值



■ 代码层实现策略

- (1) 插入并行赋值 (idealized semantics) : 在 IR 层面, 可以把 ϕ 看作所有赋值同时发生:

$x3 = \phi(x1, x2)$

if from pred1: $x3 := x1$

if from pred2: $x3 := x2$

- (2) 寄存器重命名 / 移动指令: 在真实机器代码生成时, 常见做法是在合流点前, 插入 **move** 指令, 保证合流点有正确的版本:

```
x1 = 1  
goto L3
```

```
L2:  
x2 = 2  
goto L3
```

```
L3:  
x3 =  $\phi(x1, x2)$   
y1 = x3 + 3
```

```
L1:  
x1 = 1  
goto L3_pre
```

```
L2:  
x2 = 2  
goto L3_pre
```

```
L3_pre:  
x3 = (pred == L1 ? x1 : x2) ; 插入 move  
goto L3
```

```
L3:  
y1 = x3 + 3
```



静态单赋值



■ 代码层实现策略

○ (3) LLVM 中的实现

- LLVM 在其 “InsertCopiesForPHIs” 阶段，会为每个 ϕ 在每个前驱生成一个 move（放在前驱的末尾或拆分后的插入块）。它使用与上面类似的并行复制分解算法，并分配临时 virtual registers。随后，register allocator 会尽量为不冲突的版本分配同一寄存器以消除复制（coalescing）。
- GCC (GIMPLE) 也有类似流程：GIMPLE 使用 SSA 表示，最终生成 RTL 时会做 ϕ 展开并插入复制



静态单赋值



■ ϕ 函数在循环中的功能

```
int factorial(int n) {  
    int result = 1;  
    while (n > 1) {  
        result = result * n;  
        n = n - 1;  
    }  
    return result;  
}
```

初始化`result=1`，然后在`while`循环中迭代更新`result`和`n`，直到`n<=1`。这是一个典型的后测试循环（`do-while`语义，但用`while`实现），迭代变量`n`和`result`在循环中被更新。

```
# Entry  
result = 1  
goto L_header
```

```
# Header  
L_header:  
t1 = n > 1  
if t1 goto L_body else goto L_exit
```

```
# Body  
L_body:  
t2 = result * n  
result = t2  
t3 = n - 1  
n = t3  
goto L_header
```

```
# Exit  
L_exit:  
return result
```

```
Entry: result = 1  
      goto Header
```

```
Header: if (n > 1) goto Body  
      else goto Exit
```

```
Body: result = result * n  
      n = n - 1  
      goto Header
```

```
Exit: return result
```

非SSA形式的三地址码



静态单赋值



■ ϕ 函数在循环中的功能

```
int factorial(int n) {
    int result = 1;
    while (n > 1) {
        result = result * n;
        n = n - 1;
    }
    return result;
}
```

初始化`result=1`，然后在`while`循环中迭代更新`result`和`n`，直到`n<=1`。这是一个典型的后测试循环（`do-while`语义，但用`while`实现），迭代变量`n`和`result`在循环中被更新。

```
# Entry
result_0 = 1
n_0 = n # 假设输入参数n
goto L_header
```

```
# Header
L_header:
result_1 =  $\phi$ (result_0, result_2) # 从Entry取result_0，从Body取result_2
n_1 =  $\phi$ (n_0, n_2) # 从Entry取n_0，从Body取n_2
t1 = n_1 > 1
if t1 goto L_body else goto L_exit
```

```
# Body
L_body:
t2 = result_1 * n_1
result_2 = t2
t3 = n_1 - 1
n_2 = t3
goto L_header
```

```
# Exit
L_exit:
result_3 =  $\phi$ (result_1) # 从Header取（无其他路径）
return result_3
```

在`while`循环中，迭代变量（如`n`和`result`）有“初始值”（从Entry）和“更新值”（从Body反馈）。 ϕ 在Header处合并它们：第一次迭代用初始值（`n_1 = n_0`），后续迭代用更新值（`n_1 = n_2`）

在SSA中，每个变量只赋值一次，使用下标表示版本（e.g., `result_0`, `result_1`）。 ϕ 函数置于合并点（Header块开头），形式为`x_i =  $\phi$ (x_j, x_k, ...)`，根据前驱块动态选择值。

注意：在实际SSA（如LLVM IR）中， ϕ 函数指定前驱：e.g., `result_1 =  $\phi$  [result_0, Entry], [result_2, Body]`。这里简化表示。



控制流语句翻译



- **if-else**语句，**while**语句
- 需要将语句的翻译和布尔表达式的翻译结合在一起
- 布尔表达式是被用作语句中改变控制流的条件表达式，通常用来
 - 改变控制流。布尔表达式的值由程序到达的某个位置隐含地指出。
 - 计算逻辑值。可以使用带有逻辑运算符的三地址指令进行求值。
- 布尔表达式的使用意图要根据其语法上下文确定
 - 跟在关键字**if**后面的表达式用来改变控制流
 - 一个赋值语句右部的表达式用来计算一个逻辑值
 - 可以使用两个不同的非终结符号或其它方法来区分这两种使用



控制流语句翻译



■ 布尔表达式

- 将布尔运算符作用在布尔变量或关系表达式上，构成布尔表达式
- 引入新的非终结符号B表示布尔表达式
- 布尔运算符: **&&** 、 **||** 、 **!**
- 关系表达式 **E1 rel E2**
- 关系运算符: **<**、**<=**、**=**、**!=**、**>**、**>=**
- 其中布尔运算符**&&**和**||**是左结合的，优先级**||**最低，其次是**&&**，**!**最高
- 表示为
$$B \rightarrow B \mid B \mid B \ \&\& \ B \mid ! \ B \mid (B) \mid E \ \text{rel} \ E \mid \text{true} \mid \text{false}$$



控制流语句翻译



■ 布尔表达式的高效求值

- $B1 \parallel B2$, $B1$ 为真, 则不用求 $B2$ 也能断定整个表达式为真
- $B1 \&\& B2$, $B1$ 为假, 则整个表达式肯定为假
- 如果某些程序设计语言允许这种高效的求值方式, 则编译器可以优化布尔表达式的求值过程, 只要已经求值部分足以确定整个表达式的值就可以了。

```
if x < 100 goto L2
ifFalse x > 200 goto L1
ifFalse x != y goto L1
L2:  x = 0
L1:
```

■ 短路（跳转）代码

- 布尔运算符 $\&\&$ 、 $\parallel$ 、 $!$ 被翻译成跳转指令。由跳转位置隐含的指出布尔表达式的值。
- $\text{if}(x < 100 \parallel x > 200 \&\& x \neq y) x = 0;$

图 6-34 跳转代码



控制流语句翻译



■ 语句及文法

- B 和 S 有综合属性 $code$ ，表示翻译得到的三地址代码。
- B 的继承属性 $true$ 和 $false$ ， S 的继承属性 $next$ ，表示跳转的位置。
- $S.true$ 是一个地址，用于存放了 B 为真时控制流转向的指令标号， $S.false$ 同理
- $S.next$ 也是一

$S \rightarrow \text{if } (B) S_1$

的指令标号

$S \rightarrow \text{if } (B) S_1 \text{ else } S_2$

$S \rightarrow \text{while } (B) S_1$

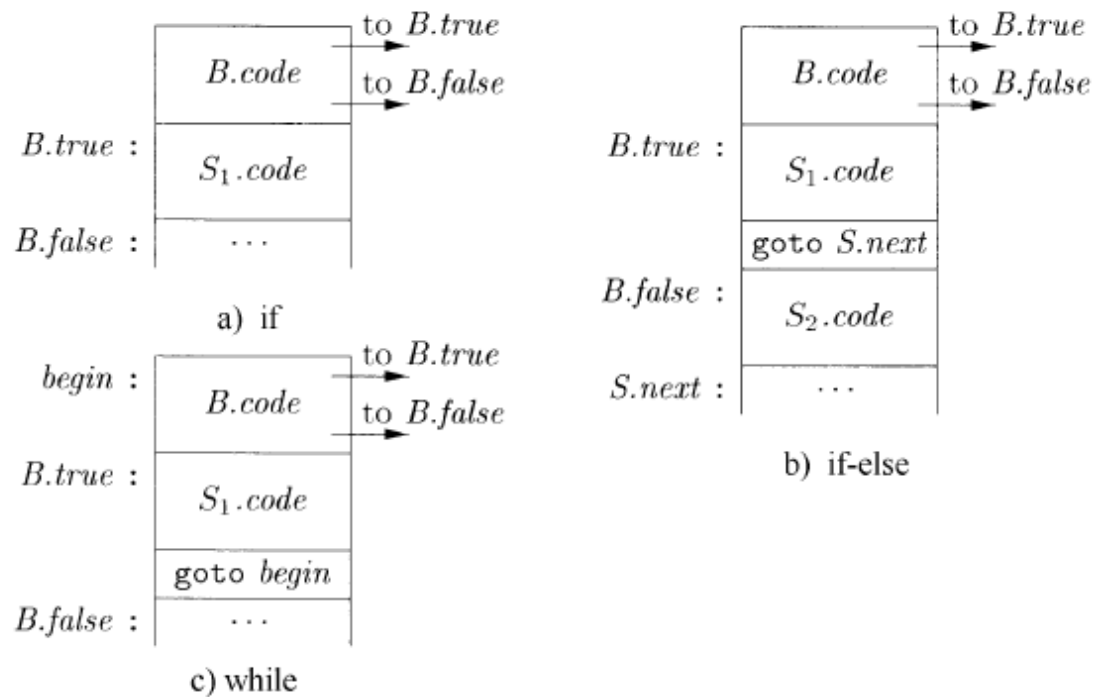


控制流语句翻译



- 语句及文法
$$S \rightarrow \text{if } (B) S_1$$
$$S \rightarrow \text{if } (B) S_1 \text{ else } S_2$$
$$S \rightarrow \text{while } (B) S_1$$

- B 和 S 有综合属性 $code$ ，表示翻译得到的三地址代码。
- B 的继承属性 $true$ 和 $false$ ， S 的继承属性 $next$ ，表示跳转的位置。





控制流语句翻译



■ 几个核心函数回顾:

- **newlabel** 函数: 生成一个用于存放标号的临时变量L, 返回变量地址;
- **label(X)**函数: 将下一条指令的标号复制给参数X;
- 翻译 $S \rightarrow \text{if} (B) S_1$, 创建B.true标号, 并指向S1的第一条指令。
- 翻译 $S \rightarrow \text{if} (B) S_1 \text{ else } S_2$, B为真时, 跳转到S1代码的第一条指令; 当B为假时跳转到S2代码的第一条指令。然后, 控制流从S1或S2转到紧跟在S的代码后面的三地址指令, 该指令由继承属性S.next指定。
- **while**语句中有个begin局部变量
-

产生式	语义规则
$P \rightarrow S$	$S.next = \text{newlabel}()$ $P.code = S.code \parallel \text{label}(S.next)$
$S \rightarrow \text{assign}$	$S.code = \text{assign.code}$
$S \rightarrow \text{if} (B) S_1$	$B.true = \text{newlabel}()$ $B.false = S_1.next = S.next$ $S.code = B.code \parallel \text{label}(B.true) \parallel S_1.code$
$S \rightarrow \text{if} (B) S_1 \text{ else } S_2$	$B.true = \text{newlabel}()$ $B.false = \text{newlabel}()$ $S_1.next = S_2.next = S.next$ $S.code = B.code$ $\quad \parallel \text{label}(B.true) \parallel S_1.code$ $\quad \parallel \text{gen('goto' } S.next)$ $\quad \parallel \text{label}(B.false) \parallel S_2.code$
$S \rightarrow \text{while} (B) S_1$	$begin = \text{newlabel}()$ $B.true = \text{newlabel}()$ $B.false = S.next$ $S_1.next = begin$ $S.code = \text{label}(begin) \parallel B.code$ $\quad \parallel \text{label}(B.true) \parallel S_1.code$ $\quad \parallel \text{gen('goto' } begin)$
$S \rightarrow S_1 S_2$	$S_1.next = \text{newlabel}()$ $S_2.next = S.next$ $S.code = S_1.code \parallel \text{label}(S_1.next) \parallel S_2.code$



控制流语句翻译



■ 布尔表达式的控制流翻译及分析:

- 布尔表达式**B**被翻译成三地址指令, 生成的条件或无条件转移指令反映**B**的值。
- **B**→**E1 rel E2**, 直接翻译成三地址比较指令, 跳转到正确位置。
- **B**→**B1 || B2**, 如果**B1**为真, **B**一定为真, 所以**B1.true**和**B.true**相同。如果**B1**为假, 那就要对**B2**求值。因此**B1.false**指向**B2**的代码开始的位置。**B2**的真假出口分别等于**B**的真假出口。

○

产生式	语义规则
$B \rightarrow B_1 B_2$	$B_1.true = B.true$ $B_1.false = newlabel()$ $B_2.true = B.true$ $B_2.false = B.false$ $B.code = B_1.code label(B_1.false) B_2.code$
$B \rightarrow B_1 \&\& B_2$	$B_1.true = newlabel()$ $B_1.false = B.false$ $B_2.true = B.true$ $B_2.false = B.false$ $B.code = B_1.code label(B_1.true) B_2.code$
$B \rightarrow ! B_1$	$B_1.true = B.false$ $B_1.false = B.true$ $B.code = B_1.code$
$B \rightarrow E_1 \text{ rel } E_2$	$B.code = E_1.code E_2.code$ $ gen('if' E_1.addr \text{ rel.op } E_2.addr 'goto' B.true)$ $ gen('goto' B.false)$
$B \rightarrow \text{true}$	$B.code = gen('goto' B.true)$
$B \rightarrow \text{false}$	$B.code = gen('goto' B.false)$



控制流语句翻译



产生式	语义规则
$P \rightarrow S$	$S.next = newlabel()$ $P.code = S.code \parallel label(S.next)$
$S \rightarrow \text{assign}$	$S.code = \text{assign}.code$
$S \rightarrow \text{if} (B) S_1$	$B.true = newlabel()$ $B.false = S_1.next = S.next$ $S.code = B.code \parallel label(B.true) \parallel S_1.code$
$S \rightarrow \text{if} (B) S_1 \text{ else } S_2$	$B.true = newlabel()$ $B.false = newlabel()$ $S_1.next = S_2.next = S.next$ $S.code = B.code$ $\parallel label(B.true) \parallel S_1.code$ $\parallel gen('goto' S.next)$ $\parallel label(B.false) \parallel S_2.code$
$S \rightarrow \text{while} (B) S_1$	$begin = newlabel()$ $B.true = newlabel()$ $B.false = S.next$ $S_1.next = begin$ $S.code = label(begin) \parallel B.code$ $\parallel label(B.true) \parallel S_1.code$ $\parallel gen('goto' begin)$
$S \rightarrow S_1 S_2$	$S_1.next = newlabel()$ $S_2.next = S.next$ $S.code = S_1.code \parallel label(S_1.next) \parallel S_2.code$

产生式	语义规则
$B \rightarrow B_1 \parallel B_2$	$B_1.true = B.true$ $B_1.false = newlabel()$ $B_2.true = B.true$ $B_2.false = B.false$ $B.code = B_1.code \parallel label(B_1.false) \parallel B_2.code$
$B \rightarrow B_1 \&\& B_2$	$B_1.true = newlabel()$ $B_1.false = B.false$ $B_2.true = B.true$ $B_2.false = B.false$ $B.code = B_1.code \parallel label(B_1.true) \parallel B_2.code$
$B \rightarrow ! B_1$	$B_1.true = B.false$ $B_1.false = B.true$ $B.code = B_1.code$
$B \rightarrow E_1 \text{ rel } E_2$	$B.code = E_1.code \parallel E_2.code$ $\parallel gen('if' E_1.addr \text{ rel } op E_2.addr 'goto' B.true)$ $\parallel gen('goto' B.false)$
$B \rightarrow \text{true}$	$B.code = gen('goto' B.true)$
$B \rightarrow \text{false}$	$B.code = gen('goto' B.false)$



控制流语句翻译



■ 布尔表达式及控制流语句翻译示例：

○ 布尔表达式翻译, $a < b$

if $a < b$ goto B.true

goto B.false

控制流语句翻译 if $(x < 100 \parallel x > 200 \&\& x \neq y)$ $x = 0$;

```
        if x < 100 goto L2
        goto L3
L3:    if x > 200 goto L4
        goto L1
L4:    if x != y goto L2
        goto L1
L2:    x = 0
L1:
```



控制流语句翻译



■ 布尔表达式及控制流语句翻译示例：

○ 布尔表达式翻译, $a < b$

if $a < b$ goto B.true

goto B.false

控制流语句翻译 if $(x < 100 \parallel x > 200 \&\& x \neq y)$ $x = 0$;

```
        if x < 100 goto L2
        goto L3
L3:    if x > 200 goto L4
        goto L1
L4:    if x != y goto L2
        goto L1
L2:    x = 0
L1:
```

思考，是否有什么问题？



控制流语句翻译



■ 避免冗余的goto指令:

- 在上面的例子中goto L3是冗余的

- X>200翻译成

```
if x > 200 goto L4
goto L1
```

L₄: ...

- 可以替换成

```
ifFalse x > 200 goto L1
L4: ...
```
- 减少了一条goto指令
- 引入一个特殊标号“fall”(穿越, fall through), 表示不要生成任何跳转指令。
- S→if (B) S1的新语义规则
$$B.true = fall$$
$$B.false = S_1.next = S.next$$
$$S.code = B.code || S_1.code$$
- 对于if-else和while语句的规则也将B.true设为fall



控制流语句翻译



- 在布尔表达式中，`&&` 与 `||` 需要 短路求值：
 - `B1 && B2`: 若 `B1` 为假，则直接判定为假，不再计算 `B2`
 - `B1 || B2`: 若 `B1` 为真，则直接判定为真，不再计算 `B2`
- 翻译时：
 - 通过 真假链 (`true/false lists`) 表示控制流
 - 利用 `fall-through` (顺序进入下一条指令) 来避免生成多余跳转
 - 穿越：
 - 对 `&&`: `false` 直接短路, `true fall-through`
 - 对 `||`: `true` 直接短路, `false fall-through`



控制流语句翻译



- 核心问题：在SDT中，我们通过附加到文法产生式的语义动作（**semantic actions**）来生成中间代码。对于表达式（如 $a = b + c$ ），翻译是线性的（自底向上或自顶向下）。但对于控制流语句（如**if-else**、**while**），存在“未知跳转目标”的问题：
 - 条件分支：**if**条件真时跳到**then**部分，否则跳到**else**或下一语句。但在解析时，**else**或循环结束的地址未知
 - 循环：**while**的条件检查需跳回循环头或跳出循环，但循环体长度在解析时不确定



控制流语句翻译



- 标准SDT是“单遍”翻译（one-pass），但控制流引入“前向引用”（forward references），即跳转到尚未生成的代码位置。如果不处理，会导致代码生成中断或错误
- 解决方案：引入回填（backpatching）技术。它允许先生成带有“占位符”（placeholders）的跳转指令，然后在后续解析中“回填”实际地址或标签
- 回填的优势：保持SDT的效率，支持LALR(1)等单遍分析器；便于处理嵌套控制流



控制流语句翻译



- 回填：回填是一种延迟绑定技术，在SDT中用于处理控制流语句的跳转指令。具体来说：
 - 先生成不完整的三地址码跳转指令（如if t goto _ 或 goto _），其中“_”是占位符（实际实现中用链表或列表存储待回填位置）
 - 当解析到目标位置时，回溯并填充这些占位符为实际标签（labels, 如L1、L2）



控制流语句翻译



- 回填：为布尔表达式和控制流语句生成目标代码的关键问题：某些跳转指令应该跳转到哪里
 - 例如：if (B) S
 - 按照短路代码的翻译方法，B的代码中有一些跳转指令在B为假时执行，
 - 这些跳转指令的目标应该跳过S对应的代码,生成这些指令时，S的代码尚未生成，因此目标不确定
 - 通过语句的继承属性next来传递。需要第二趟处理
 - 基本思想
 - 记录B的代码中跳转指令goto S.next, if ... goto S.next的位置，但是不生成跳转目标
 - 这些位置被记录到B的综合属性B.falseList中
 - 当S.next的值已知时（即S的代码生成完毕时），把S.nextList中的所有指令的目标都填上这个值
 - 回填技术
 - 生成跳转指令时暂时不指定跳转目标标号，而是使用列表记录这些不完整的指令
 - 等知道正确的目标时再填写目标标号
 - 每个列表中的指令都指向同一个目标



控制流语句翻译



■ 布尔表达式的回填翻译

- 布尔表达式用于语句的控制流时，它总是在取值true时和取值false时分别跳转到某个位置
- 引入两个综合属性记录B的代码中跳转指令goto S.next, if ... goto S.next的位置，但是不生成跳转目标
 - truelist: 包含跳转指令（位置）的列表，这些指令在取值true时执行
 - falselist: 包含跳转指令（位置）的列表，这些指令在取值false时执行
- 辅助函数
 - Makelist(i): 创建一个只包含i的列表
 - Merge(p1,p2): 将p1和p2指向的列表合并
 - Backpatch(p,i): 将i作为目标标号插入到p所指列表中的各指令中
- 语义属性：在SDT文法中，为非终结符添加属性如{code, truelist, falselist, nextlist}



控制流语句翻译



- 我们看一个具体的例子！

```
if ((x < y) && (y < z) || (x == z))
```

```
    a = 1;
```

```
else
```

```
    a = 2;
```

- 翻译过程（带穿越）

- 表达式 $(x < y) \ \&\& \ (y < z)$

- 如果 $x < y$ 为假 $\rightarrow$ 整体为假（短路），跳转到 **false**

- 如果 $x < y$ 为真 $\rightarrow$ fall-through, 进入 $y < z$ 判断

- 与 $(x == z)$ 的 $\parallel$

- 如果 $(x < y \ \&\& \ y < z)$ 为真 $\rightarrow$ 整体为真（短路），跳转到 **true**

- 否则继续判断 $(x == z)$



控制流语句翻译



■ 我们看一个具体的例子！

```
if ((x < y) && (y < z) || (x == z))  
    a = 1;  
else  
    a = 2;
```

■ 翻译过程（带穿越）

- `gen("if a rel b goto _")` // quad i → truelist = {i}
- `gen("goto _")` // quad i+1 → falselist = {i+1}

对 B1 && B2:

1. 生成 B1 → 得到 B1.true, B1.false
2. M = nextquad(), backpatch(B1.true, M) (若 B1 为真, 跳到 B2)
3. 生成 B2 → B.true = B2.true, B.false = merge(B1.false, B2.false)

对 B1 || B2:

1. 生成 B1 → B1.true, B1.false
2. M = nextquad(), backpatch(B1.false, M) (若 B1 为假, 跳到 B2)
3. 生成 B2 → B.true = merge(B1.true, B2.true), B.false = B2.false



控制流语句翻译



■ 我们看一个具体的例子！

```
if ((x < y) && (y < z) || (x == z))  
    a = 1;  
else
```

■ 翻译过程（带穿越）

○ 表达式 $(x < y) \ \&\& \ (y < z)$

- 如果 $x < y$ 为假 $\rightarrow$ 整体为假（短路），跳转到 **false**
- 如果 $x < y$ 为真 $\rightarrow$ fall-through，进入 $y < z$ 判断

○ 与 $(x == z)$ 的 $\parallel$

- 如果 $(x < y \ \&\& \ y < z)$ 为真 $\rightarrow$ 整体为真（短路），跳转到 **true**
- 否则继续判断 $(x == z)$

$(x < y)$

true $\rightarrow$ fall-through 到 if $y < z$

false $\rightarrow$ 跳到 $(x == z)$

$(y < z)$

true $\rightarrow$ 直接跳到 then 部分

false $\rightarrow$ fall-through 到 $(x == z)$

$(x == z)$

true $\rightarrow$ 直接跳到 then 部分

false $\rightarrow$ 跳到 else 部分



控制流语句翻译

```
if ((x < y) && (y < z) || (x == z))  
    a = 1;  
else  
    a = 2;
```



- 我们看一个具体的例子！
- 完整思考流程：
 - 初始化 `nextquad = 1`。我们按上面的模板生成代码并记录各个 list
 - (1) 生成 `x < y`（记作 B1）
 - 1: if `x < y` goto _ // B1.true = {1}
 - 2: goto _ // B1.false = {2}
 - **nextquad = 3**
 - (2) 要生成 `B1 && B2`，先把 B1.true 指向 B2 开始位置 `M = nextquad = 3`
 - `backpatch(B1.true, 3)` // 设置 quad1 目标为 3
 - (3) 生成 `y < z`（记作 B2），从 quad 3 开始
 - 3: if `y < z` goto _ // B2.true = {3}
 - 4: goto _ // B2.false = {4}
 - **nextquad = 5**

现在 `B_and = B1 && B2`：
`B_and.true = B2.true = {3}`
`B_and.false = merge(B1.false, B2.false) = {2,4}`



控制流语句翻译

```
if ((x < y) && (y < z) || (x == z))  
    a = 1;  
else  
    a = 2;
```



- 我们看一个具体的例子！
- 完整思考流程：
 - 初始化 `nextquad = 1`。我们按上面的模板生成代码并记录各个 list
 - (4) 处理 `(B_and) || (x == z)`：对 `||`，需要把左侧的 `falselist` 指向右侧开始 `M2 = nextquad = 5`
 - `backpatch(B_and.false, 5)` // 设置 `quad2` 和 `quad4` 的目标为 5

现在 `B_and = B1 && B2`：
`B_and.true = B2.true = {3}`
`B_and.false = merge(B1.false, B2.false) = {2,4}`



控制流语句翻译

```
if ((x < y) && (y < z) || (x == z))  
    a = 1;  
else  
    a = 2;
```



- 我们看一个具体的例子！
- 完整思考流程：
 - 初始化 `nextquad = 1`。我们按上面的模板生成代码并记录各个 list
 - (4) 处理 `(B_and) || (x == z)`：对 `||`，需要把左侧的 `falselist` 指向右侧开始 `M2 = nextquad = 5`
 - `backpatch(B_and.false, 5)` // 设置 `quad2` 和 `quad4` 的目标为 5
 - (5) 生成右侧 `x == z`（记作 **C**），从 `quad 5` 开始
 - 5: `if x == z goto _` // `C.true = {5}`
 - 6: `goto _` // `C.false = {6}`
 - `nextquad = 7`



控制流语句翻译

```
if ((x < y) && (y < z) || (x == z))  
    a = 1;  
else  
    a = 2;
```



■ 我们看一个具体的例子！

■ 完整思考流程：

- 初始化 `nextquad = 1`。我们按上面的模板生成代码并记录各个 list

- (6) 生成 `then / else` 代码并回填

- 将 `then` 语句 (`a = 1`) 放在 `nextquad = 7`:

- `backpatch(overall.true, 7)` // 把 `quad3` 和 `quad5` 的目标填为 7

- 7: `a = 1`

- 8: `goto _` // after-then 跳过 `else` (占位, 稍后 `backpatch` 到 `end`)

- **`nextquad = 9`**

- 9: `a = 2` // `else` 分支

- **`nextquad = 10`**

- 回填 `else (false)` 目标及 `then` 后跳转:

- `backpatch(overall.false, 9)` // 把 `quad6` 的目标设为 9 (`else` 起始)

- `backpatch({8}, 10)` // 把 `quad8` 的目标设为 10 (`then` 后跳转到 `end`)

- 最后 `nextquad = 10` 是程序续处 (`end label`)



控制流语句翻译

```
if ((x < y) && (y < z) || (x == z))  
    a = 1;  
else  
    a = 2;
```



- 我们看一个具体的例子！
- 最终填好的三地址码

```
1: if x < y goto 3  
2: goto 5  
3: if y < z goto 7  
4: goto 5  
5: if x == z goto 7  
6: goto 9  
7: a = 1  
8: goto 10  
9: a = 2  
10: ... // 后续代码
```

说明：

1 的 true 目标是 3（进入 $y < z$ ）；
2/4 都跳到 5（当 $x < y$ 或 $y < z$ 为假时，进入右侧 $x == z$ 的判断）；
3 和 5 都跳到 7（当任一判断为真时进入 then）；
6 跳到 9（若右侧 $x == z$ 为假则进入 else）；
8 跳到 10（then 执行完跳过 else）。



控制流语句翻译

```
if ((x < y) && (y < z) || (x == z))  
    a = 1;  
else  
    a = 2;
```



- 我们看一个具体的例子！
- 最终填好的三地址码

```
1: if x < y goto 3  
2: goto 5  
3: if y < z goto 7  
4: goto 5  
5: if x == z goto 7  
6: goto 9  
7: a = 1  
8: goto 10  
9: a = 2  
10: ... // 后续代码
```

说明：

1 的 true 目标是 3（进入 $y < z$ ）；
2/4 都跳到 5（当 $x < y$ 或 $y < z$ 为假时，进入右侧 $x == z$ 的判断）；
3 和 5 都跳到 7（当任一判断为真时进入 then）；
6 跳到 9（若右侧 $x == z$ 为假则进入 else）；
8 跳到 10（then 执行完跳过 else）。

思考：这个是不是最优解？



控制流语句翻译

```
if ((x < y) && (y < z) || (x == z))  
    a = 1;  
else  
    a = 2;
```



- 我们看一个具体的例子！
- 最终填好的三地址码

```
1: if x < y goto 3  
2: goto 5  
3: if y < z goto 7  
4: goto 5  
5: if x == z goto 7  
6: goto 9  
7: a = 1  
8: goto 10  
9: a = 2  
10: ... // 后续代码
```

```
1: if x < y goto 3  
2: if x == z goto 7  
3: if y < z goto 7  
4: a = 2  
5: goto 8  
7: a = 1  
8: ...
```

说明：

1 的 true 目标是 3（进入 $y < z$ ）；
2/4 都跳到 5（当 $x < y$ 或 $y < z$ 为假时，进入右侧 $x == z$ 的判断）；
3 和 5 都跳到 7（当任一判断为真时进入 then）；
6 跳到 9（若右侧 $x == z$ 为假则进入 else）；
8 跳到 10（then 执行完跳过 else）。

思考：这个是不是最优解？



控制流语句翻译



■ 总结！翻译过程（带穿越）

○ 表达式 $(x < y) \ \&\& \ (y < z)$

- 如果 $x < y$ 为假 $\rightarrow$ 整体为假（短路），跳转到 **false**
- 如果 $x < y$ 为真 $\rightarrow$ fall-through，进入 $y < z$ 判断

○ 与 $(x == z)$ 的 `||`

- 如果 $(x < y \ \&\& \ y < z)$ 为真 $\rightarrow$ 整体为真（短路），跳转到 **true**
- 否则继续判断 $(x == z)$

$(x < y)$
true $\rightarrow$ fall-through 到 if $y < z$
false $\rightarrow$ 跳到 $(x == z)$

$(y < z)$
true $\rightarrow$ 直接跳到 then 部分
false $\rightarrow$ fall-through 到 $(x == z)$

$(x == z)$
true $\rightarrow$ 直接跳到 then 部分
false $\rightarrow$ 跳到 else 部分



控制流语句翻译



- 引入：为什么需要“穿越”？
 - 在控制流语句（如 **if-else**、**while**、**for**）中，中间代码往往包含跳转指令
 - 这些跳转的目标位置（**label**）在生成代码时通常还未知，需要在整个语法分析完成后才能确定
 - 为了处理这种情况，我们引入 穿越（**translation schemes with inherited attributes + backpatching**），在语义动作中延迟确定目标地址



控制流语句翻译



■ “穿越”的基本思想

- 穿越（**traversal/translation scheme**）：在语法制导翻译（**SDT**）中，利用继承属性，将未确定的跳转目标传递下去，直到生成目标点时再进行“回填”
- 配合 回填（**backpatching**）：将之前生成的跳转指令列表与目标语句地址绑定



控制流语句翻译



■ “穿越”的基本思想

- 穿越（**traversal/translation scheme**）：在语法制导翻译（**SDT**）中，利用继承属性，将未确定的跳转目标传递下去，直到生成目标点时再进行“回填”
- 配合 回填（**backpatching**）：将之前生成的跳转指令列表与目标语句地址绑定

换句话说：穿越 = 在语法树遍历过程中传递未完成的跳转信息



控制流语句翻译



■ 利用“穿越”修改布尔表达式的语义规则：

```
test = E1.addr rel.op E2.addr  
  
s = if B.true ≠ fall and B.false ≠ fall then  
    gen('if' test 'goto' B.true) || gen('goto' B.false)  
    else if B.true ≠ fall then gen('if' test 'goto' B.true)  
    else if B.false ≠ fall then gen('ifFalse' test 'goto' B.false)  
    else ''  
  
B.code = E1.code || E2.code || s
```

图 6-39 $B \rightarrow E_1 \text{ rel } E_2$ 的语义规则

```
B1.true = if B.true ≠ fall then B.true else newlabel()  
B1.false = fall  
B2.true = B.true  
B2.false = B.false  
B.code = if B.true ≠ fall then B1.code || B2.code  
        else B1.code || B2.code || label(B1.true)
```

图 6-40 $B \rightarrow B_1 || B_2$ 的语义规则

- 注意B.true=fall时，还得为B₁.true new一个label

```
B1.true = B.true  
B1.false = newlabel()  
B2.true = B.true  
B2.false = B.false  
B.code = B1.code || label(B1.false) || B2.code
```




控制流语句翻译



■ $B \rightarrow B_1 \&\& B_2$ 带“穿越”的语义规则:

$B_1.false = \text{if } (B.false = \text{fall}) \text{ newlabel() else } B.false$

$B_1.true = \text{fall}$

$B_2.true = B.true$

$B_2.false = B.false$

$B.code = \text{if } (B.false = \text{fall}) \text{ then } B_1.code || B_2.code || \text{label } (B_1.false) \text{ else } B_1.code || B_2.code$

$$\begin{array}{l|l} B \rightarrow B_1 \&\& B_2 & \begin{array}{l} B_1.true = \text{newlabel()} \\ B_1.false = B.false \\ B_2.true = B.true \\ B_2.false = B.false \\ B.code = B_1.code || \text{label}(B_1.true) || B_2.code \end{array} \end{array}$$



控制流语句翻译



■ 使用标号fall的控制流语句翻译示例：

- if (x<100 || x>200 && x!=y) x=0;

```
B1.true = if B.true ≠ fall then B.true else newlabel()
B1.false = fall
B2.true = B.true
B2.false = B.false
B.code = if B.true ≠ fall then B1.code || B2.code
        else B1.code || B2.code || label(B1.true)
```

$B \rightarrow B_1 || B_2$ 的语义规则

```
test = E1.addr rel.op E2.addr
s = if B.true ≠ fall and B.false ≠ fall then
    gen('if' test 'goto' B.true) || gen('goto' B.false)
    else if B.true ≠ fall then gen('if' test 'goto' B.true)
    else if B.false ≠ fall then gen('ifFalse' test 'goto' B.false)
    else ''
```

$B.code = E_1.code || E_2.code || s$

$B \rightarrow E_1 \text{ rel } E_2$ 的语义规则

```
B.true = fall
B.false = S1.next = S.next
S.code = B.code || S1.code
```

```
S.next = newlabel()
P.code = S.code || label(S.next)
```

```
if x < 100 goto L2
ifFalse x > 200 goto L1
ifFalse x != y goto L1
L2: x = 0
L1:
```



控制流语句翻译



- 布尔表达式有如下两个功能:
- 改变控制流, 跳转
 - 刚刚前面重点讨论, 用非终结符号**B**表示此种功能的布尔表达式以示区别
- 求值
 - 如 $x=true$, $x=a<b$
- 统一处理

$S \rightarrow \text{id} = E; \mid \text{if} (E) S \mid \text{while} (E) S \mid S S$

$E \rightarrow E \mid \mid E \mid E \&\& E \mid E \text{ rel } E \mid E + E \mid (E) \mid \text{id} \mid \text{true} \mid \text{false}$

- 使用不同的代码生成函数处理表达式的两种角色。 $E.n$ 对应于抽象语法树上的表达式节点。两个函数:
 - **jump**, 对于出现在 $S \rightarrow \text{while}(E)S1$ 中的 E , 调用 $E.n.\text{jump}(t,f)$
 - **rvalue**, 对于出现在 $S \rightarrow \text{id}=E$ 中的 E , 在节点 $E.n$ 上调用 **rvalue**。如果 E 是算术表达式, 按照算术表达式的翻译生成代码。如果 E 是布尔表达式, 如 $E1\&E2$, 首先为 E 生成跳转代码, 然后在跳转代码的真假出口分别将 **true** 或 **false** 赋给一个新的临时变量 t 。



控制流语句翻译



■ 示例

- 赋值语句 $x = a < b \ \&\& \ c < d$

```
ifFalse a < b goto L1
ifFalse c < d goto L1
t = true
goto L2
L1:  t = false
L2:  x = t
```

图 6-42 通过计算一个临时变量的值来翻译一个布尔类型的赋值语句



控制流语句翻译



■ 布尔表达式的回填翻

- 1) $B \rightarrow B_1 \ || \ M \ B_2$ { $backpatch(B_1.falselist, M.instr);$
 $B.truelist = merge(B_1.truelist, B_2.truelist);$
 $B.falselist = B_2.falselist; \}$
- 2) $B \rightarrow B_1 \ \&\& \ M \ B_2$ { $backpatch(B_1.truelist, M.instr);$
 $B.truelist = B_2.truelist;$
 $B.falselist = merge(B_1.falselist, B_2.falselist); \}$
- 3) $B \rightarrow ! B_1$ { $B.truelist = B_1.falselist;$
 $B.falselist = B_1.truelist; \}$
- 4) $B \rightarrow (B_1)$ { $B.truelist = B_1.truelist;$
 $B.falselist = B_1.falselist; \}$
- 5) $B \rightarrow E_1 \ \mathbf{rel} \ E_2$ { $B.truelist = makelist(nextinstr);$
 $B.falselist = makelist(nextinstr + 1);$
 $gen('if' \ E_1.addr \ \mathbf{rel.op} \ E_2.addr \ 'goto \ -');$
 $gen('goto \ -');$ }

- 6) $B \rightarrow \mathbf{true}$ { $B.truelist = makelist(nextinstr);$
 $gen('goto \ -');$ }

- 7) $B \rightarrow \mathbf{false}$ { $B.falselist = makelist(nextinstr);$
 $gen('goto \ -');$ }

- 8) $M \rightarrow \epsilon$ { $M.instr = nextinstr; \}$



控制流语句翻译



■ 布尔表达式的回填翻

```
backpatch(list p, label L) {  
    for each position i in p {  
        code[i] = "goto " + L; // 回填  
    }  
}
```

- 1) $B \rightarrow B_1 \ || \ M \ B_2$ { $backpatch(B_1.falselist, M.instr);$
 $B.truelist = merge(B_1.truelist, B_2.truelist);$
 $B.falselist = B_2.falselist; \}$
- 2) $B \rightarrow B_1 \ \&\& \ M \ B_2$ { $backpatch(B_1.truelist, M.instr);$
 $B.truelist = B_2.truelist;$
 $B.falselist = merge(B_1.falselist, B_2.falselist); \}$
- 3) $B \rightarrow ! B_1$ { $B.truelist = B_1.falselist;$
 $B.falselist = B_1.truelist; \}$
- 4) $B \rightarrow (B_1)$ { $B.truelist = B_1.truelist;$
 $B.falselist = B_1.falselist; \}$
- 5) $B \rightarrow E_1 \ \text{rel} \ E_2$ { $B.truelist = makelist(nextinstr);$
 $B.falselist = makelist(nextinstr + 1);$
 $gen('if' \ E_1.addr \ rel.op \ E_2.addr \ 'goto \ -');$
 $gen('goto \ -');$ }
- 6) $B \rightarrow \text{true}$ { $B.truelist = makelist(nextinstr);$
 $gen('goto \ -');$ }
- 7) $B \rightarrow \text{false}$ { $B.falselist = makelist(nextinstr);$
 $gen('goto \ -');$ }
- 8) $M \rightarrow \epsilon$ { $M.instr = nextinstr; \}$



控制流语句翻译



■ 布尔表达式的回填翻译

$$B \rightarrow B_1 \ || \ B_2 \quad \left\{ \begin{array}{l} B_1.true = B.true \\ B_1.false = newlabel() \\ B_2.true = B.true \\ B_2.false = B.false \\ B.code = B_1.code \ || \ label(B_1.false) \ || \ B_2.code \end{array} \right.$$

$$B \rightarrow B_1 \ || \ M \ B_2 \quad \left\{ \begin{array}{l} backpatch(B_1.falselist, M.instr); \\ B.truelist = merge(B_1.truelist, B_2.truelist); \\ B.falselist = B_2.falselist; \end{array} \right. \}$$



控制流语句翻译



■ 布尔表达式的回填翻译

$$B \rightarrow B_1 \ \&\& \ B_2 \quad \left| \begin{array}{l} B_1.true = newlabel() \\ B_1.false = B.false \\ B_2.true = B.true \\ B_2.false = B.false \\ B.code = B_1.code \ || \ label(B_1.true) \ || \ B_2.code \end{array} \right.$$

$$B \rightarrow B_1 \ \&\& \ M \ B_2 \quad \{ \begin{array}{l} backpatch(B_1.truelist, M.instr); \\ B.truelist = B_2.truelist; \\ B.falselist = merge(B_1.falselist, B_2.falselist); \end{array} \}$$



控制流语句翻译



■ 布尔表达式的回填翻译

$$B \rightarrow ! B_1$$

$$\left| \begin{array}{l} B_1.true = B.false \\ B_1.false = B.true \\ B.code = B_1.code \end{array} \right.$$

$$B \rightarrow ! B_1$$

$$\left\{ \begin{array}{l} B.truelist = B_1.falselist; \\ B.falselist = B_1.truelist; \end{array} \right\}$$

$$B \rightarrow (B_1)$$

$$\left\{ \begin{array}{l} B.truelist = B_1.truelist; \\ B.falselist = B_1.falselist; \end{array} \right\}$$



控制流语句翻译



■ 布尔表达式的回填翻译

$$B \rightarrow E_1 \text{ rel } E_2 \quad \left| \quad \begin{array}{l} B.\text{code} = E_1.\text{code} \parallel E_2.\text{code} \\ \parallel \text{gen('if' } E_1.\text{addr rel.op } E_2.\text{addr 'goto' } B.\text{true}) \\ \parallel \text{gen('goto' } B.\text{false}) \end{array} \right.$$

$$B \rightarrow E_1 \text{ rel } E_2 \quad \{ \begin{array}{l} B.\text{truelist} = \text{makelist}(\text{nextinstr}); \\ B.\text{falselist} = \text{makelist}(\text{nextinstr} + 1); \\ \text{emit('if' } E_1.\text{addr rel.op } E_2.\text{addr 'goto' } -'); \\ \text{emit('goto' } -); \end{array} \}$$



控制流语句翻译



■ 布尔表达式的回填翻译

$B \rightarrow \text{true}$ $B.\text{code} = \text{gen}(\text{'goto' } B.\text{true})$

$B \rightarrow \text{false}$ $B.\text{code} = \text{gen}(\text{'goto' } B.\text{false})$

$B \rightarrow \text{true}$ $\{ B.\text{truelist} = \text{makelist}(\text{nextinstr});$
 $\text{emit}(\text{'goto -'}); \}$

$B \rightarrow \text{false}$ $\{ B.\text{falselist} = \text{makelist}(\text{nextinstr});$
 $\text{emit}(\text{'goto -'}); \}$

$M \rightarrow \epsilon$ $\{ M.\text{instr} = \text{nextinstr}; \}$



控制流语句翻译



■ 回填和非回填方法的比较

$B \rightarrow B_1 \parallel B_2$	$B_1.true = B.true$ $B_1.false = newlabel()$ $B_2.true = B.true$ $B_2.false = B.false$ $B.code = B_1.code \parallel label(B_1.false) \parallel B_2.code$
-----------------------------------	--

1) $B \rightarrow B_1 \parallel M B_2$ { $backpatch(B_1.falselist, M.instr);$
 $B.truelist = merge(B_1.truelist, B_2.truelist);$
 $B.falselist = B_2.falselist; }$

- 应为相应的list的赋值或者merge
- 生成label的地方，在回填方案中使用M来记录相应的代码位置，M.inst需对应label的标号
- 原方案生成的指令goto B1.false，现在生成了goto M.inst



控制流语句翻译



■ 回填和非回填方法的比较

$B \rightarrow E_1 \text{ rel } E_2$	$B.code = E_1.code \parallel E_2.code$ $\parallel \text{gen('if' } E_1.addr \text{ rel.op } E_2.addr \text{ 'goto' } B.true)$ $\parallel \text{gen('goto' } B.false)$
--------------------------------------	---

5) $B \rightarrow E_1 \text{ rel } E_2$ { $B.truelist = \text{makelist}(\text{nextinstr});$
 $B.falselist = \text{makelist}(\text{nextinstr} + 1);$
 $\text{gen('if' } E_1.addr \text{ rel.op } E_2.addr \text{ 'goto' -});$
 $\text{gen('goto' -});$ }

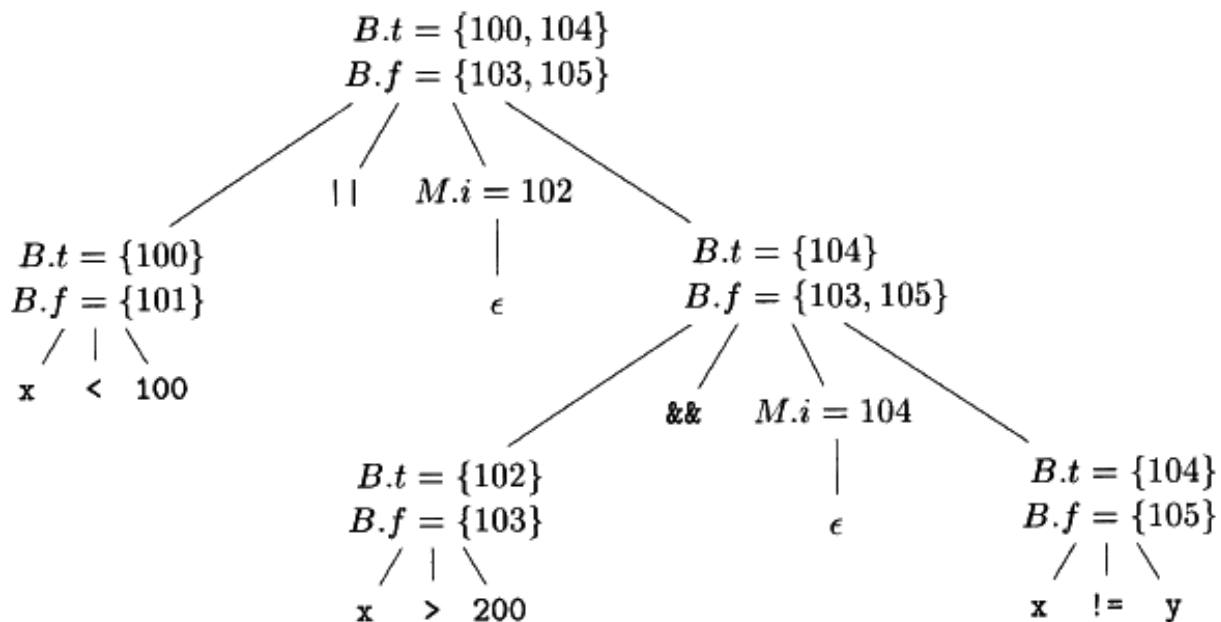
- 回填时生成指令坯，然后加入相应的list
- 原来跳转到B.true的指令，现在被加入到B.truelist中



控制流语句翻译

■ 布尔表达式的回填例子

○ $x < 100 \parallel x > 200 \ \&\& \ x \neq y$



```

100:  if x < 100 goto -
101:  goto -
102:  if x > 200 goto 104
103:  goto -
104:  if x != y goto -
105:  goto -
  
```

a) 将 104 回填到指令 102 中之后

```

100:  if x < 100 goto -
101:  goto 102
102:  if x > 200 goto 104
103:  goto -
104:  if x != y goto -
105:  goto -
  
```

b) 将 102 回填到指令 101 中之后

```

    if x < 100 goto L2
    goto L3
L3:  if x > 200 goto L4
    goto L1
L4:  if x != y goto L2
    goto L1
L2:  x = 0
L1:
  
```





控制流语句翻译



■ 控制转移语句的回填

- $S \rightarrow \text{if}(B) S \quad |$
 $\text{if}(B) S \text{ else } S \quad |$
 $\text{while}(B) S \quad |$
 $\{ L \} \quad | A$

$L \rightarrow L S \quad | S$

○ 语句的综合属性: nextlist

- nextlist中的跳转指令的目标应该是S执行完毕之后紧接着执行的下一条指令的位置
- 考虑S是while语句、if语句的子语句时, 分别应该跳转到哪里



控制流语句翻译



■ 控制转移语句的回填

1) $S \rightarrow \text{if}(B) M S_1 \{ \text{backpatch}(B.\text{truelist}, M.\text{instr});$
 $S.\text{nextlist} = \text{merge}(B.\text{falselist}, S_1.\text{nextlist}); \}$

2) $S \rightarrow \text{if}(B) M_1 S_1 N \text{ else } M_2 S_2$
 $\{ \text{backpatch}(B.\text{truelist}, M_1.\text{instr});$
 $\text{backpatch}(B.\text{falselist}, M_2.\text{instr});$
 $\text{temp} = \text{merge}(S_1.\text{nextlist}, N.\text{nextlist});$
 $S.\text{nextlist} = \text{merge}(\text{temp}, S_2.\text{nextlist}); \}$

6) $M \rightarrow \epsilon \quad \{ M.\text{instr} = \text{nextinstr}; \}$

7) $N \rightarrow \epsilon \quad \{ N.\text{nextlist} = \text{makelist}(\text{nextinstr});$
 $\text{gen}(\text{'goto -'}); \}$

- M的作用就是用M.instr记录下一个指令的位置
 - 规则1中记录了then分支的代码起始位置;
 - 规则2中, 分别记录了then分支和else分支的起始位置;
- N的作用是生成goto指令, N.nextlist只包含这个指令的位置



控制流语句翻译



■ 控制转移语句的回填

- 3) $S \rightarrow \text{while } M_1 (B) M_2 S_1$
- ```
{ backpatch($S_1.nextlist$, $M_1.instr$);
 backpatch($B.truelist$, $M_2.instr$);
 $S.nextlist = B.falselist$;
 gen('goto' $M_1.instr$); }
```
- 4)  $S \rightarrow \{ L \}$       {  $S.nextlist = L.nextlist$ ; }
- 5)  $S \rightarrow A ;$       {  $S.nextlist = \text{null}$ ; }
- 8)  $L \rightarrow L_1 M S$       { backpatch( $L_1.nextlist$ ,  $M.instr$ );  
                             $L.nextlist = S.nextlist$ ; }
- 9)  $L \rightarrow S$       {  $L.nextlist = S.nextlist$ ; }



# 控制流语句翻译



## ■ 控制转移语句的回填

$S \rightarrow \text{if} ( B ) S_1$

$B.true = \text{newlabel}()$   
 $B.false = S_1.next = S.next$   
 $S.code = B.code \parallel \text{label}(B.true) \parallel S_1.code$

$S \rightarrow \text{if} ( B ) S_1 \text{ else } S_2$

$B.true = \text{newlabel}()$   
 $B.false = \text{newlabel}()$   
 $S_1.next = S_2.next = S.next$   
 $S.code = B.code$   
 $\parallel \text{label}(B.true) \parallel S_1.code$   
 $\parallel \text{gen('goto' } S.next)$   
 $\parallel \text{label}(B.false) \parallel S_2.code$

$S \rightarrow \text{if} ( B ) M S_1 \{ \text{backpatch}(B.truelist, M.instr);$   
 $S.nextlist = \text{merge}(B.falselist, S_1.nextlist); \}$

$S \rightarrow \text{if} ( B ) M_1 S_1 N \text{ else } M_2 S_2$   
 $\{ \text{backpatch}(B.truelist, M_1.instr);$   
 $\text{backpatch}(B.falselist, M_2.instr);$   
 $temp = \text{merge}(S_1.nextlist, N.nextlist);$   
 $S.nextlist = \text{merge}(temp, S_2.nextlist); \}$



# 控制流语句翻译



## ■ 控制转移语句的回填

$S \rightarrow \text{while} ( B ) S_1$

```
begin = newlabel()
B.true = newlabel()
B.false = S.next
S1.next = begin
S.code = label(begin) || B.code
 || label(B.true) || S1.code
 || gen('goto' begin)
```

$S \rightarrow \text{while } M_1 ( B ) M_2 S_1$

```
{ backpatch(S1.nextlist, M1.instr);
 backpatch(B.truelist, M2.instr);
 S.nextlist = B.falselist;
 emit('goto' M1.instr); }
```



# 控制流语句翻译

## ■ 控制转移语句的回填

 $P \rightarrow S$ 

$$\begin{aligned} S.next &= newlabel() \\ P.code &= S.code \parallel label(S.next) \end{aligned}$$
 $S \rightarrow \text{assign}$ 

$$S.code = \text{assign.code}$$
 $S \rightarrow S_1 S_2$ 

$$\begin{aligned} S_1.next &= newlabel() \\ S_2.next &= S.next \\ S.code &= S_1.code \parallel label(S_1.next) \parallel S_2.code \end{aligned}$$
 $S \rightarrow \{ L \}$ 

$$\{ S.nextlist = L.nextlist; \}$$
 $S \rightarrow A ;$ 

$$\{ S.nextlist = \text{null}; \}$$
 $M \rightarrow \epsilon$ 

$$\{ M.instr = nextinstr; \}$$
 $N \rightarrow \epsilon$ 

$$\{ N.nextlist = makelist(nextinstr); \\ \text{emit('goto -')}; \}$$
 $L \rightarrow L_1 M S$ 

$$\{ \text{backpatch}(L_1.nextlist, M.instr); \\ L.nextlist = S.nextlist; \}$$
 $L \rightarrow S$ 

$$\{ L.nextlist = S.nextlist; \}$$



# 控制流语句翻译

```
do {
 a = a + 1;
} while (a < 10 && b > 0);
```



- 看一个do while回填的例子
- 翻译步骤（带 nextquad）



# 控制流语句翻译



## ■ 看一个do while回填的例子

```
do {
 a = a + 1;
} while (a < 10 && b > 0);
```

## ■ 条件 $a < 10 \ \&\& \ b > 0$ 需要 短路求值

## ■ 先判断 $a < 10$ ，若为假，直接退出循环；若为真，再判断 $b > 0$

$$S \rightarrow \text{while } M_1 (B) M_2 S_1$$
$$\begin{aligned} &\{ \text{backpatch}(S_1.\text{nextlist}, M_1.\text{instr}); \\ &\quad \text{backpatch}(B.\text{truelist}, M_2.\text{instr}); \\ &\quad S.\text{nextlist} = B.\text{falselist}; \\ &\quad \text{emit}(\text{'goto' } M_1.\text{instr}); \} \end{aligned}$$



# 控制流语句翻译



## ■ 看一个do while回填的例子

```
do {
 a = a + 1;
} while (a < 10 && b > 0);
```

- 条件  $a < 10 \ \&\& \ b > 0$  需要 短路求值
- 先判断  $a < 10$ ，若为假，直接退出循环；若为真，再判断  $b > 0$
- 翻译步骤（带 nextquad）

$$S \rightarrow \textbf{while } M_1 ( B ) M_2 S_1$$
$$\begin{aligned} &\{ \text{backpatch}(S_1.\text{nextlist}, M_1.\text{instr}); \\ &\quad \text{backpatch}(B.\text{truelist}, M_2.\text{instr}); \\ &\quad S.\text{nextlist} = B.\text{falselist}; \\ &\quad \text{emit('goto' } M_1.\text{instr}); \} \end{aligned}$$



# 控制流语句翻译

```
do {
 a = a + 1;
} while (a < 10 && b > 0);
```



## ■ 看一个do while回填的例子

### • 翻译步骤（带 nextquad）

○ 假设 nextquad = 1 开始

○ (1) 标记循环体起点

■  $M = \text{nextquad} = 1$

○ (2) 生成循环体

■ 1:  $t1 = a + 1$

■ 2:  $a = t1$

■  $\text{nextquad} = 3$

○ (3) 生成布尔条件  $a < 10 \ \&\& \ b > 0$

$S \rightarrow \text{while } M_1 (B) M_2 S_1$

```
{ backpatch($S_1.\text{nextlist}$, $M_1.\text{instr}$);
 backpatch($B.\text{truelist}$, $M_2.\text{instr}$);
 $S.\text{nextlist} = B.\text{falselist}$;
 emit('goto' $M_1.\text{instr}$); }
```





# 控制流语句翻译

```
do {
 a = a + 1;
} while (a < 10 && b > 0);
```



## ■ 看一个do while回填的例子

### • 翻译步骤（带 nextquad）

#### ○ (3) 生成布尔条件 $a < 10 \ \&\& \ b > 0$ ➤ (4) 回填

■ 先处理  $a < 10$ :

■ 3: if  $a < 10$  goto 5 // 真的情况继续判断  $b > 0$

■ 4: goto \_ // 假的情况退出

■ 此时:

■  $B1.true = \{3\}$

■  $B1.false = \{4\}$

■ 再处理  $b > 0$ （从 nextquad=5 开始）:

■ 5: if  $b > 0$  goto \_

■ 6: goto \_

■ backpatch( $B1.true$ , 5)（真分支跳到判断  $b > 0$ ）

■ backpatch( $B2.true$ ,  $M=1$ )（ $b > 0$  成立时，跳回循环体起点）

■  $B.false = \{4, 6\} \rightarrow$  填为循环出口。



# 控制流语句翻译

```
do {
 a = a + 1;
} while (a < 10 && b > 0);
```



## ■ 看一个do while回填的例子

### • 翻译步骤（带 nextquad）

#### ○ (3) 生成布尔条件 $a < 10 \ \&\& \ b > 0$ ➤ (4) 回填

■ 先处理  $a < 10$ :

■ 3: if  $a < 10$  goto 5 // 真的情况继续判断  $b > 0$

■ 4: goto \_ // 假的情况退出

■ 此时:

■  $B1.true = \{3\}$

■  $B1.false = \{4\}$

■ 再处理  $b > 0$ （从 nextquad=5 开始）:

■ 5: if  $b > 0$  goto \_

■ 6: goto \_

■ backpatch( $B1.true$ , 5)（真分支跳到判断  $b > 0$ ）

■ backpatch( $B2.true$ ,  $M=1$ )（ $b > 0$  成立时，跳回循环体起点）

■  $B.false = \{4, 6\} \rightarrow$  填为循环出口。

1:  $t1 = a + 1$

2:  $a = t1$

3: if  $a < 10$  goto 5

4: goto 7 // false branch of  $a < 10 \rightarrow$  exit

5: if  $b > 0$  goto 1

6: goto 7 // false branch of  $b > 0 \rightarrow$  exit

7: ... // 后续语句



# Break、Continue的处理



- 虽然break、continue在语法上是一个独立的句子，但是它的代码和外围语句相关
- 方法：(break语句)
  - 跟踪外围语句S，
  - 生成一个跳转指令坯
  - 将这个指令坯的位置加入到S的nextlist中
- 跟踪的方法
  - 在符号表中设置break条目，令其指向外围语句
  - 在符号表中设置指向S的nextlist的指针，然后把这个指令坯的位置直接加入到nextlist中



# 中间代码表示



## 五、简答题（共 1 题，共 10 分）

1. (5 分) 在图 1 中间代码翻译方案中添加处理如下控制流构造的规则（注意布尔表达式之前的否运算）：

$S \rightarrow \text{for } (S_1; !B; S_2) S_3$

2. (5 分) 结合图 2 布尔表达式的翻译方案，翻译如下代码（第 5 到第 7 行）：

要求：画出语法分析树，并在每个节点上标注相应的属性，给出最终的中间代码。

```
(1) int i;
(2) int n = 100;
(3) int sum = 0;
(4)
(5) for (i = 0; i < n && sum < 1000; i = i + 1) {
(6) sum = sum + i;
(7) }
```

|     |  |
|-----|--|
| 得分  |  |
| 评分人 |  |

| 产生式                                                  | 语义规则                                                                                                                                                                                                                                                                         |
|------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| $P \rightarrow S$                                    | $S.next = \text{newlabel}()$<br>$P.code = S.code \parallel \text{label}(S.next)$                                                                                                                                                                                             |
| $S \rightarrow \text{assign}$                        | $S.code = \text{assign}.code$                                                                                                                                                                                                                                                |
| $S \rightarrow \text{if } (B) S_1$                   | $B.true = \text{newlabel}()$<br>$B.false = S_1.next = S.next$<br>$S.code = B.code \parallel \text{label}(B.true) \parallel S_1.code$                                                                                                                                         |
| $S \rightarrow \text{if } (B) S_1 \text{ else } S_2$ | $B.true = \text{newlabel}()$<br>$B.false = \text{newlabel}()$<br>$S_1.next = S_2.next = S.next$<br>$S.code = B.code$<br>$\parallel \text{label}(B.true) \parallel S_1.code$<br>$\parallel \text{gen}('goto' S.next)$<br>$\parallel \text{label}(B.false) \parallel S_2.code$ |
| $S \rightarrow \text{while } (B) S_1$                | $begin = \text{newlabel}()$<br>$B.true = \text{newlabel}()$<br>$B.false = S.next$<br>$S_1.next = begin$<br>$S.code = \text{lable}(begin) \parallel B.code$<br>$\parallel \text{label}(B.true) \parallel S_1.code$<br>$\parallel \text{gen}('goto' begin)$                    |
| $S \rightarrow S_1 S_2$                              | $S_1.next = \text{newlable}()$<br>$S_2.next = S.next$<br>$S.code = S_1.code \parallel \text{label}(S_1.next) \parallel S_2.code$                                                                                                                                             |



# 中间代码表示



## 五、简答题（共 1 题，共 10 分）

1.（5 分）在图 1 中间代码翻译方案中添加处理如下控制流构造的规则（注意布尔表达式之前的否运算）：

$$S \rightarrow \text{for}(S_1; !B; S_2) S_3$$

2.（5 分）结合图 2 布尔表达式的翻译方案，翻译如下代码（第 5 到第 7 行）：

要求：画出语法分析树，并在每个节点上标注相应的属性，给出最终的中间代码。

```
(1) int i;
(2) int n = 100;
(3) int sum = 0;
(4)
(5) for (i = 0; i < n && sum < 1000; i = i + 1) {
(6) sum = sum + i;
(7) }
```

| 产生式                                                |
|----------------------------------------------------|
| $P \rightarrow S$                                  |
| $S \rightarrow \text{assign}$                      |
| $S \rightarrow \text{if}(B) S_1$                   |
| $S \rightarrow \text{if}(B) S_1 \text{ else } S_2$ |
| $S \rightarrow \text{while}(B) S_1$                |
| $S \rightarrow S_1 S_2$                            |

## 参考答案：

1.

`begin = new_label()`

`B.false = new_label()`

`B.true = S.next()`

`S1.next = begin`

`S2.next = begin`

`S3.next = newlabel()`

```
S.code = S1.code || label(begin) || B.code
 || label(B.false) || S3.code ||
 label(S3.next)
 || S2.code || gen('goto' begin)
```



# 中间代码表示

## 五、简答题（共 1 题，共 10 分）

1. (5 分) 在图 1 中间代码翻译方案中添加处理如下控制流构造的规则（注意布尔表达式之前的否运算）：

$S \rightarrow \text{for}(S_1; !B; S_2) S_3$

2. (5 分) 结合图 2 布尔表达式的翻译方案，翻译如下代码（第 5 到第 7 行）：

要求：画出语法分析树，并在每个节点上标注相应的属性，给出最终的中间代码。

```
(1) int i;
(2) int n = 100;
(3) int sum = 0;
(4)
(5) for (i = 0; i < n && sum < 1000; i = i + 1) {
(6) sum = sum + i;
(7) }
```

| 产生式                                                |
|----------------------------------------------------|
| $P \rightarrow S$                                  |
| $S \rightarrow \text{assign}$                      |
| $S \rightarrow \text{if}(B) S_1$                   |
| $S \rightarrow \text{if}(B) S_1 \text{ else } S_2$ |
| $S \rightarrow \text{while}(B) S_1$                |
| $S \rightarrow S_1 S_2$                            |

2.

$i = 0$

L1: if  $i < 0$  goto L2

goto L5

L2: if  $\text{sum} < 1000$  goto L3

goto L5

L3:  $\text{sum} = \text{sum} + i$

goto L4

L4:  $i = i + 1$

goto L1

L5:

